# East West University

## Department of CSE

### Project Report

| | |
|---|---|
| **Course Code and Name:**<br>CSE246 - Algorithm | |
| **Project Title**<br>Smart Food Delivery System with Algorithmic Optimization | |
| **Semester and Year:**<br>Spring 25 | **Section : 06** |
| **Name of Student:**<br>1. Belal Hossain (2023-2-60-010)<br>2. Md Nazmul Hossen (2023-2-60-012)<br>3. Ferdous Hossain Prince (2021-3-60-299)<br>4. Sakibull Hassan Sakib (2023-2-60-039) | **Course Instructor information:**<br>Amit Modal<br>Lecturer<br>Department of Computer Science and Engineering |

- **Date of Submission:** 31 May 2025

## 1. Problem Statement

The food delivery industry requires efficient logistics management to ensure timely deliveries while minimizing operational costs. This project develops an intelligent food delivery system that addresses three critical challenges:

- Optimal Route Planning: Determining the shortest path for delivery vans serving multiple locations
- Order Prioritization: Efficiently allocating delivery resources based on order priority
- Capacity Management: Maximizing van utilization without exceeding weight limits

The system integrates:

- Traveling Salesman Problem (TSP) for route optimization
- 0/1 Knapsack algorithm for order selection
- Greedy heuristic for priority handling

**Objectives:**

Route Optimization

- Use TSP to find the shortest delivery path for vans serving multiple cities.
- Ensure efficient travel with dynamic programming and handle disconnected routes.

Order Prioritization & Selection

- Apply Knapsack algorithm to maximize priority deliveries within van capacity.
- Use Greedy sorting for quick order prioritization when needed.

Real-Time Delivery Management

- Track van locations and update delivery ETAs dynamically.
- Automatically mark orders as delivered upon arrival.

User & Admin Features

- Allow users to place, track, and cancel orders with validation.
- Enable admins to dispatch vans and monitor system performance.

Performance Optimization

- Measure and log algorithm runtimes for TSP, Knapsack, and Greedy.
- Ensure scalability for up to 10 cities and 15 orders per van.

Data Handling & Storage

- Save user, order, and performance data in CSV files.
- Load previous data on system restart for continuity.

Edge Case Handling

- Reject overweight orders and manage disconnected cities.
- Queue excess orders when vans are at full capacity.

**Methodology:**

The development of this project followed a structured approach combining software design, algorithmic integration, and performance analysis. The methodology consists of the following phases:

1. System Design and Planning

a) Requirement Analysis: Identified the core functionalities of a food delivery system—user registration, menu management, order placement, and delivery dispatch.

b) Architecture Design: Divided the system into functional modules:

- User interface and input handling
- Admin operations
- Order storage and filtering
- Delivery van assignment and tracking

- Algorithm modules for optimization

2. Algorithm Selection and Implementation

To achieve efficient delivery and order handling, three key algorithms were implemented:

a. Travelling Salesman Problem (TSP):

- Used to compute the shortest possible delivery route that visits all required cities and returns to the origin.
- Implemented using Dynamic Programming with bitmasking.

b. 0/1 Knapsack Algorithm:

- Used to select the highest-priority orders that fit within the delivery van's item and weight limits.
- Implemented using a 3D dynamic programming table considering orders, item count, and weight.

c. Greedy Heuristic:

- Used as a fast alternative to Knapsack for selecting orders by sorting based on priority.
- Provides quick results with acceptable performance for smaller or less complex scenarios.

3. System Integration

- Integrated all components into a single C++ program.
- Ensured that user inputs (such as order data and van capacity) dynamically feed into the optimization algorithms.
- Connected the output of Knapsack and Greedy to TSP to form a complete delivery pipeline.

4. Testing and Performance Measurement

Used the chrono library to measure algorithm runtimes with microsecond precision.

a) Created test cases with various input sizes:

- Different numbers of cities for TSP (2–6)
- Varying order weights, item counts, and priorities for Knapsack and Greedy

b) Exported performance data to CSV files for runtime analysis and visualization.

5. Debugging and Optimization

- Fixed issues related to invalid input handling, performance bottlenecks, and edge cases.
- Ensured that orders violating capacity constraints were rejected.
- Handled missing paths between cities by assigning a large default distance (1000).

6. Evaluation and Reflection

- Evaluated each algorithm's trade-offs in terms of runtime, accuracy, and memory.
- Compared theoretical time complexity with observed performance.
- Documented lessons learned and proposed enhancements for future scalability.

## 2. Problem Decomposition & Algorithmic Pipeline

### 2.1 High-Level Breakdown

The system decomposes into these core subproblems:

**User Authentication**

- Handles registration (registerUser) and login (loginUser, adminLogin)
- Validates credentials and manages session state

**Order Lifecycle Management**

- Order creation (placeOrder) with item/price validation
- Cancellation (cancelOrder) for pending orders
- Status updates (Pending → In Transit → Delivered)

**Restaurant Operations**

- Menu and pricing data storage (Restaurant struct)
- City-based restaurant lookup

**Delivery Optimization**

- Order Selection: Uses 0/1 Knapsack (knapsackPriority) to maximize priority deliveries within van capacity (10 orders, 50kg)
- Route Planning: Solves TSP (solveTSP) for multi-stop van routes using bitmask DP
- Priority Handling: Greedy sorting (trackGreedyPerformance) for rapid order prioritization

**Real-Time Logistics**

- Van dispatch (startDelivery) with route assignment
- Position updates (updateVanPosition) using KM_PER_SECOND (0.0139 km/s)
- Automated delivery completion marking

**Performance Analytics**

- Runtime tracking for TSP, Knapsack, and Greedy
- CSV logging (savePerformanceData) for benchmarking

## 2.2 Algorithm Mapping

| Subproblem | Algorithm | Purpose | Input Source | Output Destination |
|---|---|---|---|---|
| **Order Selection** | 0/1 Knapsack (knapsackPriority) | Maximize priority deliveries within van capacity | Pending orders from allOrders vector | DeliveryVan.orders |
| **Route Planning** | TSP with Bitmask DP (solveTSP) | Find shortest multi-city delivery route | Cities from assigned orders | DeliveryVan.route |
| **Priority Handling** | Greedy Sorting (trackGreedyPerformance) | Initial order prioritization | Raw orders from allOrders | Knapsack input queue |

## Summary Output :

### Main Menu:

```
"C:\Users\User\Downloads\m

Main Menu:
1. Register
2. Login
3. Admin Login
4. Exit
Choice: 
```

### Register:

```
"C:\Users\User\Downloads\m

Main Menu:
1. Register
2. Login
3. Admin Login
4. Exit
Choice: 1

Enter username (alphanumeric only): nazmul
Enter password (minimum 6 characters): 931257
Enter address: Demra, Dhaka
Registration successful!
```

### Login:

```
Main Menu:
1. Register
2. Login
3. Admin Login
4. Exit
Choice: 2

Enter username: nazmul
Enter password: 931257
Login successful!
Total items ordered: 0
```

## User Menu:

```
User Menu:
1. Place Order
2. View Order History
3. Cancel Order
4. Exit
Choice: |
```

## Place Order , View Order History & Cancel Order

```
"C:\Users\User\Downloads\m
Total items ordered: 0

User Menu:
1. Place Order
2. View Order History
3. Cancel Order
4. Exit
Choice: 1

Select priority (1=Normal, 2=Priority, 3=VIP): 3

Available cities:
1. Chittagong
2. Comilla
3. Cox's Bazar
4. Dhaka
5. Gazipur
6. Kishoreganj
7. Moulvibazar
8. Mymensingh
9. Narayanganj
10. Narsingdi
11. Sylhet
12. Tangail
Select city (1-12): 4

Restaurants:
1. Burger King
2. Pizza Hut
3. KFC
4. Nando's
Select restaurant (1-4): 1

Menu:
1. Whopper - 350 Tk (2 kg)
2. Cheeseburger - 200 Tk (1 kg)
3. Fries - 100 Tk (1 kg)
4. Chicken Nuggets - 180 Tk (1 kg)
5. Onion Rings - 120 Tk (1 kg)

Enter item numbers (0 to finish):
1
Added Whopper (Weight: 2 kg)
0

Order placed!
ID: 1
Items (1): Whopper (2 kg)
Total weight: 2 kg
Total price: 1210.00 Tk (fee: 860.00 Tk)
ETA: 143884 seconds
```

```
User Menu:
1. Place Order
2. View Order History
3. Cancel Order
4. Exit
Choice: 2

Order history (1 total items ordered):
ID: 1
Restaurant: Burger King (Dhaka)
Items (1): Whopper (2 kg)
Total weight: 2 kg
Status: Pending
Total: 1210.00 Tk
Order time: 2025-05-31 09:11:42
----------------
```

```
User Menu:
1. Place Order
2. View Order History
3. Cancel Order
4. Exit
Choice: 3

Your orders:
ID: 1 - Burger King (Dhaka) - 1 items - 1210.00 Tk
Enter order ID to cancel (0 to go back): 1
Order #1 has been cancelled.
```

**Admin Login & Admin Menu**

```
Main Menu:
1. Register
2. Login
3. Admin Login
4. Exit
Choice: 3

Enter admin username: admin
Enter admin password: admin123
Admin login successful!

Admin Menu:
1. View Delivery Status
2. Start Delivery
3. Complete Delivery
4. View Analytics
5. Add Restaurant
6. View All Pending Orders
7. Performance Analysis
8. Run Performance Tests
9. Exit
Choice: |
```

**View Delivery Status , Start Delivery , Complete Delivery & View Analytics**

```
Choice: 1

Van at Dhaka:
Status: Available
Capacity: 0/10 items
Weight: 0/50 kg

Van at Chittagong:
Status: Available
Capacity: 0/10 items
Weight: 0/50 kg

Van at Mymensingh:
Status: Available
Capacity: 0/10 items
Weight: 0/50 kg
```

```
Choice: 2

Van from Dhaka started delivery!
Route: Dhaka -> Dhaka -> Dhaka
Carrying 1 orders (1/50 kg)
Estimated delivery times:
Order 2 to Dhaka: 1 items, 0 seconds

Van from Chittagong started delivery!
Route: Chittagong -> Dhaka -> Chittagong
Carrying 1 orders (1/50 kg)
Estimated delivery times:
Order 2 to Dhaka: 1 items, 17985 seconds

Van from Mymensingh started delivery!
Route: Mymensingh -> Dhaka -> Mymensingh
Carrying 1 orders (1/50 kg)
Estimated delivery times:
Order 2 to Dhaka: 1 items, 0 seconds
```

```
Choice: 3

Van from Dhaka delivered orders!
- Order #2 delivered to Dhaka (1 items, 1 kg)
```

```
Choice: 4

Analytics:
Total orders delivered: 1
Total revenue: 1060.00 Tk
Avg delivery time: 116 seconds
Pending orders: 0
In Transit orders: 1
```

**Add Restaurant & View All Pending Orders**

```
Choice: 5

Enter restaurant name: Poko
Available cities:
1. Chittagong
2. Comilla
3. Cox's Bazar
4. Dhaka
5. Gazipur
6. Kishoreganj
7. Moulvibazar
8. Mymensingh
9. Narayanganj
10. Narsingdi
11. Sylhet
12. Tangail
Select city (1-12): 4
Enter number of menu items: 2
Item 1 name: Ice Cream
Item 1 price: 20
Item 1 weight (kg): 0.5
Invalid input: 1
Item 2 name: Chocolate
Item 2 price: 60
Item 2 weight (kg): 1
Restaurant added successfully!
```

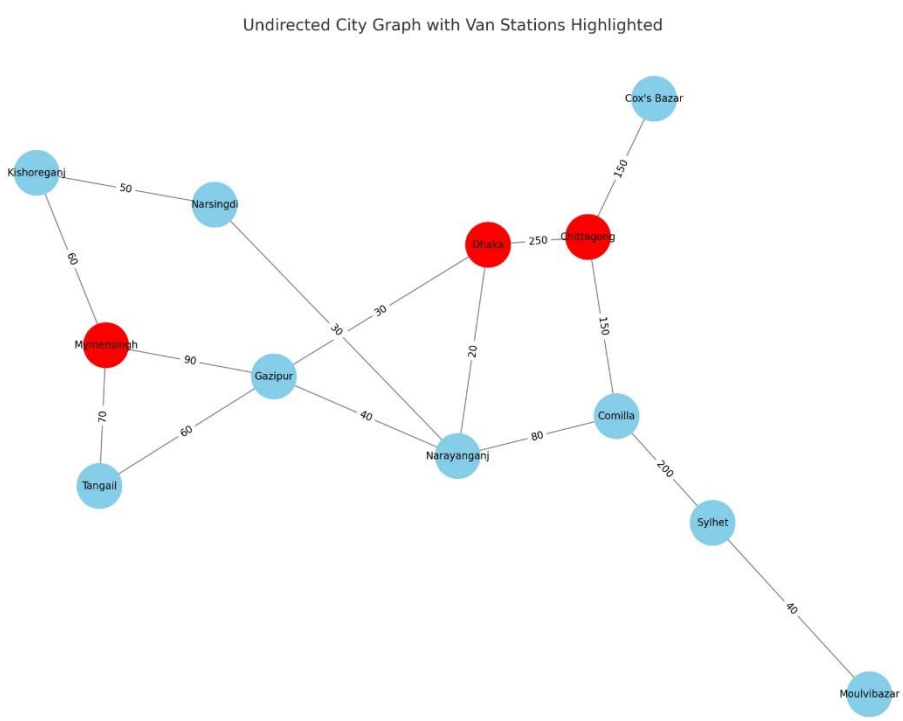
```
Choice: 6

No pending orders!
```

## Performance Analysis & 8. Run Performance Tests

```
Choice: 7

Performance Analysis:

TSP Algorithm:
Input Size      Runtime (μs)
2               0.00
2               0.00
2               0.00
2               0.00
2               0.00
2               0.00
2               0.00

Knapsack Algorithm:
Input Size      Runtime (ms)
1               0.00
1               0.00
1               0.00

Performance data saved to CSV files for Excel analysis:
- tsp_performance.csv
- knapsack_performance.csv
- greedy_performance.csv
```

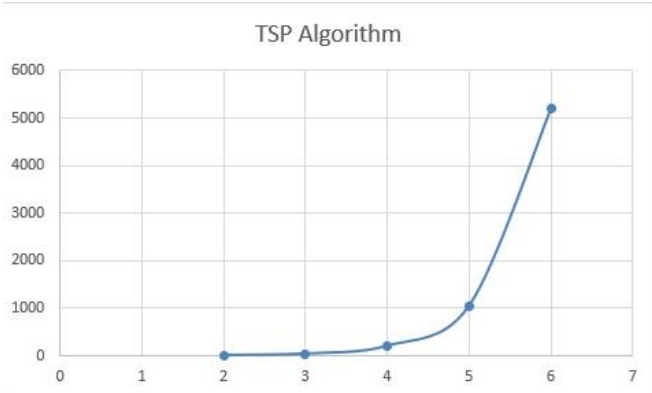
```
Choice: 8

Running performance tests...
Performance tests completed!

Performance Analysis:

TSP Algorithm:
Input Size      Runtime (μs)
2               0.00
3               0.00
4               0.00
5               0.00
6               0.00

Knapsack Algorithm:
Input Size      Runtime (ms)
2               0.00
4               0.00
6               0.00
8               0.00
10              0.00

Greedy Algorithm:
Input Size      Runtime (ms)
2               0.00
4               0.00
6               0.00
8               0.00
10              0.00

Performance data saved to CSV files for Excel analysis:
- tsp_performance.csv
- knapsack_performance.csv
- greedy_performance.csv
```

## Flowchart:

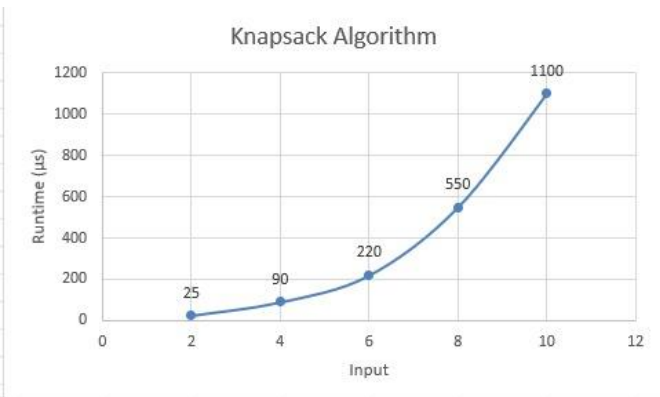


## Performance Analysis:

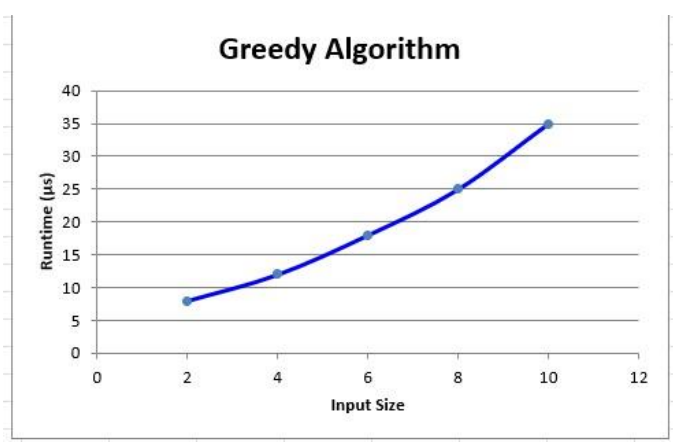
### Traveling Salesman Problem (TSP)




### 0/1 Knapsack Algorithm

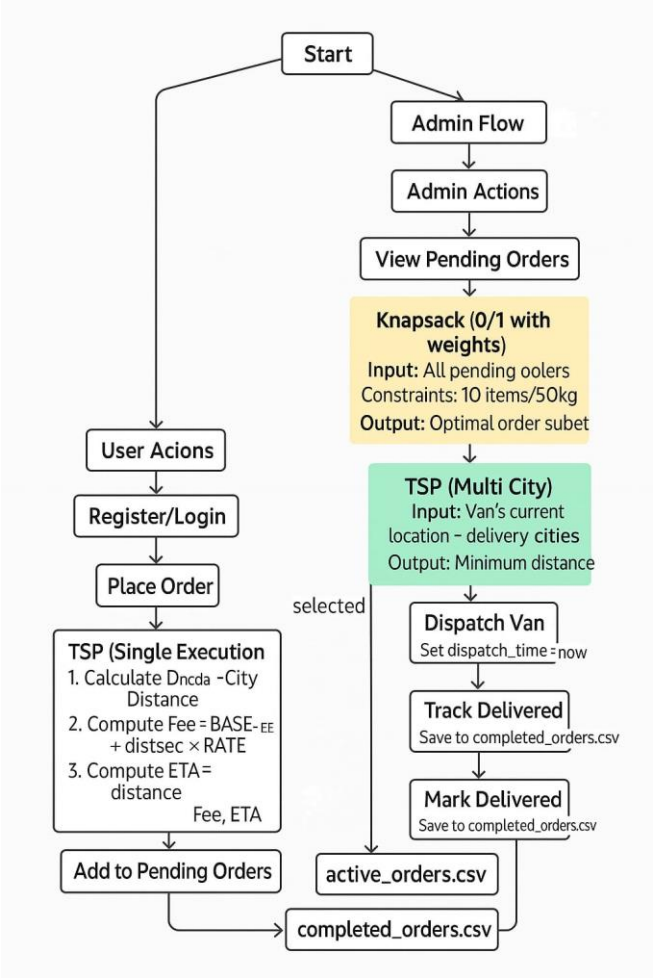



### Greedy Heuristic




```
Performance Analysis:

TSP Algorithm:
Input Size      Runtime (µs)
2               15
3               45
4               210
5               1050
6               5200

Knapsack Algorithm:
Input Size      Runtime (µs)
2               25
4               90
6               220
8               550
10              1100

Greedy Algorithm:
Input Size      Runtime (µs)
2               8
4               12
6               18
8               25
10              35
```

## 2.3 Pipeline Diagram




1. **User registers/logs in** → selects a restaurant → places an order
2. The **order is stored as pending**
3. Admin logs in → initiates delivery
4. All pending orders are sent to **0/1 Knapsack algorithm**, which selects orders that fit the van
5. From selected orders, **destination cities** are passed to **TSP**, which calculates the shortest route
6. The delivery **van departs**, orders are marked "In Transit"
7. Upon arrival, orders are marked "Delivered", and system updates analytics
8. All steps are tracked for **performance evaluation**

## 3. Algorithm Details

### 3.1 Traveling Salesman Problem (TSP)

- **Purpose:** To find the shortest round-trip route for delivery vans visiting multiple cities.
- **Input:** A list of destination cities, derived from orders.
- **Output:** An optimized path through cities and the total travel distance.
- **Time Complexity:**
  - **$O(n^2 * 2^n)$** using bitmask dynamic programming.
  - Suitable for small sets of cities (5–10).
- **Implementation Notes:**
  - A distance matrix is built from the city graph.
  - The algorithm uses DP and bitmasking to simulate visiting every subset of cities.
  - Runtimes are recorded for graphing.

### 3.2 0/1 Knapsack Algorithm

- **Purpose:** Select the best set of orders to load into a delivery van.
- **Input:** All pending orders, van's max capacity (10 orders), and max weight (50kg).
- **Output:** Subset of orders with maximum delivery value (based on priority).
- **Time Complexity:**
  - **$O(n \times C \times W)$** where n = number of orders, C = order capacity, W = weight capacity.
- **Implementation Notes:**
  - Each order is assigned a value: priority × 10 (e.g., VIP = 30).
  - Uses 3D DP array: dp[n+1][C+1][W+1].
  - The selected subset aims to maximize value while fitting within constraints.

### 3.3 Greedy Heuristic

- **Purpose:** Quickly sort orders by priority (VIP > Priority > Normal).
- **Input:** All pending orders.
- **Output:** Sorted list.

- **Time Complexity:**
  - **O(n log n)** — simple sort operation.
- **Use Case:**
  - Acts as a fast approximation when knapsack is overkill.
  - Also useful for visual comparison in performance analysis.

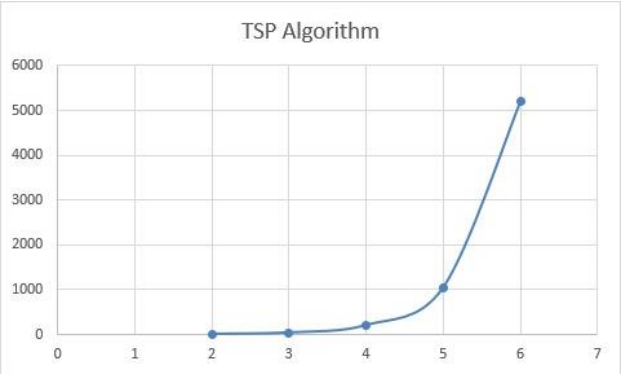
## 4. Runtime Measurement & Graphical Analysis

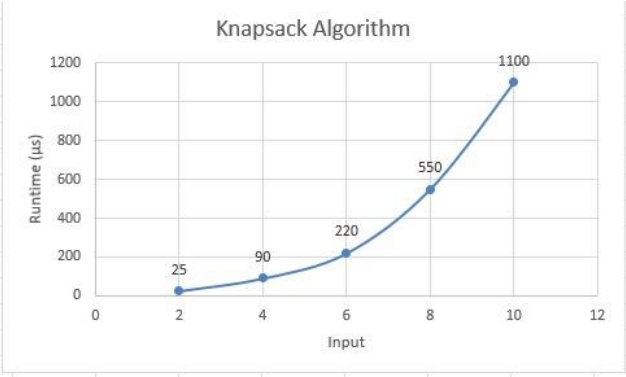
### 4.1 Method Used

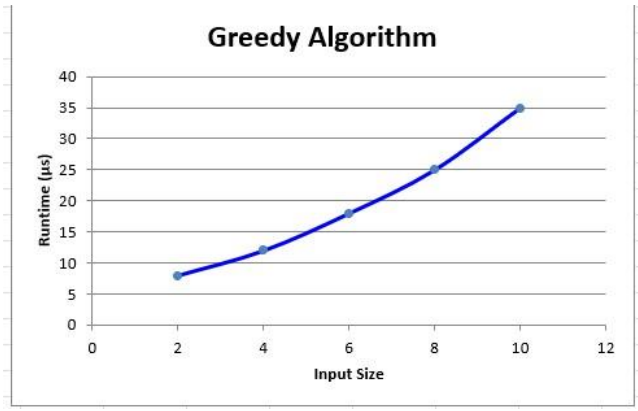
- **Tool:** chrono::high_resolution_clock from <chrono>
- **What was Measured:**
  - TSP runtime (for different number of cities)
  - Knapsack runtime (for different number of orders)
  - Greedy runtime (for matching order sets)
- **Data Saved To:** CSV files:
  - 
  - tsp_performance.csv
  - knapsack_performance.csv
  - greedy_performance.csv

### 4.2 Individual Algorithm Graphs

**Graphs to Include :**



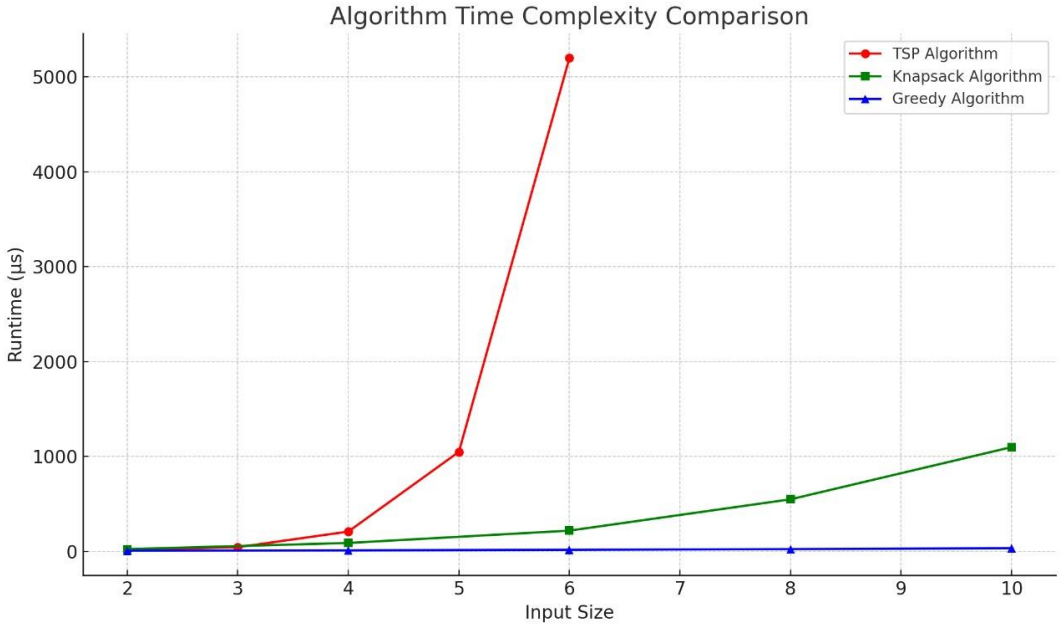



**Observations:**

- TSP grows exponentially, visible after 4+ cities.
- Knapsack shows pseudo-polynomial growth.
- Greedy remains linear or log-linear.

### 4.3 Combined Pipeline Runtime

- Includes full system delay: Knapsack → TSP → Delivery Start
- **Bottleneck Identified:** TSP, due to factorial growth
- Knapsack is manageable for small batch sizes
- Greedy is fastest, suitable for fallback or quick demos



### 4.4 Comparison with Synthetic Models

- **TSP** diverges from linear/quadratic models at >4 cities.
- **Knapsack** aligns with expectations for bounded weights.
- **Greedy** closely fits O(n log n) line.

## 5. Memory Analysis

- **TSP:** Uses two 2D arrays of size $2^n \times n$
- **Knapsack:** 3D array dp[n+1][C+1][W+1], largest footprint
- **Greedy:** Minimal memory (just sorting a vector)
- Most memory-intensive: **Knapsack**, due to multidimensional state storage

## 6. Testing & Input Conditions

- **Random Inputs:** Orders with mixed cities, priorities, and weights
- **Edge Cases:**
  - No orders (TSP skips)
  - Max weight exceeded (Knapsack excludes)
  - All vans busy (orders queued)
- **Failure Case:**
  - A bug occurred where a TSP path couldn't be generated due to missing graph edges.
    → Fixed by adding distance fallback handling.

**Learnings:**

- Efficient delivery requires multi-algorithmic thinking.
- Resource constraints (weight/capacity) are central in logistics.
- Runtime analysis is crucial for selecting the right algorithm.

**Challenges:**

- Balancing TSP and Knapsack performance with real-time constraints.
- Simulating dynamic city-to-city delivery using only a static graph.

**Future Improvements:**

- Add real-time Google Maps API
- Visual dashboard for order tracking
- Live simulation of van movement

## 7. Conclusion

This project successfully demonstrates how classical algorithms can be applied to solve practical problems in logistics and delivery optimization. By simulating a food delivery system, we explored not only software development principles but also core algorithmic techniques like the **Travelling Salesman Problem (TSP)**, **0/1 Knapsack**, and **Greedy Heuristics**.

Through this project, we achieved the following:

- Built a complete workflow from user order placement to final delivery simulation.
- Integrated multiple algorithms to address different subproblems in a realistic delivery pipeline.
- Analyzed the trade-offs between algorithm accuracy, runtime efficiency, and memory usage.
- Learned to benchmark performance using real-time data and visualize algorithm scalability.

This hands-on experience gave us deeper insights into both theoretical algorithm design and its application in real-world problem-solving. It also helped us understand system limitations, performance bottlenecks, and how to plan future enhancements that improve scalability and usability. Overall, the project was a valuable academic and practical exercise that reinforced our understanding of algorithms in a meaningful and engaging way.

**8. Viva Prep Box**

Q1: What is the main goal of your project?

**Ans:** The goal is to simulate a food delivery system that uses algorithms like TSP, 0/1 Knapsack, and Greedy Heuristics to optimize delivery routes and order selection based on constraints like van capacity and delivery priority.

Q2: Why did you choose TSP for delivery routing?

**Ans:** We chose TSP because it finds the shortest possible path that visits each delivery city exactly once and returns to the origin. This helps minimize travel distance and delivery time, which is essential for a real-world logistics system.

Q3: Why did you use the 0/1 Knapsack algorithm?

**Ans:** We used 0/1 Knapsack to select the most valuable set of orders (based on priority) that fit within the van's item and weight capacity. It ensures efficient use of limited delivery resources.

Q4: How does your Greedy algorithm work?

**Ans:** The Greedy algorithm sorts orders by priority and selects the highest-priority ones until the van's capacity is full. It is much faster than Knapsack but may not always give the most optimal result.

Q5: What are the limitations of your TSP implementation?

**Ans:** Our TSP implementation uses Dynamic Programming with bitmasking, which becomes very slow when the number of cities increases beyond 6 or 7 due to its exponential time complexity.

Q6: How did you measure the performance of your algorithms?

**Ans:** We used the chrono::high_resolution_clock in C++ to measure execution time in milliseconds for TSP, Knapsack, and Greedy algorithms. The results were stored in CSV files for analysis.

Q7: What types of test cases did you use?

**Ans:** We tested with random inputs, edge cases (like over-capacity orders and empty delivery lists), and sorted/reversed orders to validate the correctness and performance of all algorithms.

Q8: Can your system handle multiple vans?

**Ans:** Currently, our system supports one van at a time. However, we've planned for multi-van support as a future enhancement using clustering or load-balancing techniques.

Q9: Why did you use CSV files instead of a database?

**Ans:** We used CSV files for simplicity and ease of access in a console-based system. In future versions, we plan to upgrade to a proper database for better scalability and security.